Day 15: Kubernetes Fundamentals

COVER SLIDE / OVERVIEW

```
ubuntu@master-node:~/day15-kubernetes

# Deploying, Scaling & Self-Healing on K8s
$ kubectl apply -f deployment.yaml
$ kubectl scale deployment --replicas=3
$ kubectl delete pod nginx-deployment-...
* Status: Self-Healing & Interview Q&A Ready
```

MISSION ROADMAP (01 TO 07)

- ✓ **01: Concepts & Architecture**
- ✓ **02: Manifests & Deployment (Steps 1-3)**
- ✓ **03: Deep Inspection & Logs (Step 4)**
- ✓ **04: Scaling & NodePort Access (Steps 5-6)**
- ✓ **05: Troubleshooting & Self-Healing (Step 7)**
- ✓ **06: Deliverables & Checklist**
- ✓ **07: Interview Readiness Q&A**

01 / Core Concepts & Docker vs Kubernetes

Kubernetes Architectural Components

- **Cluster:** The complete set of control plane and worker nodes running workloads.
- **Control Plane:** Manages cluster state, scheduling, and controller loops.
- **Worker Node:** Runs containerized Pods managed by kubelet and container runtime.
- **Pod:** Smallest deployable unit containing one or more co-located containers.
- **Deployment:** Declares desired state, managing replica scaling and updates.
- **Service:** Provides a stable IP, DNS name, and load balancing across Pods.

Docker vs Kubernetes Comparison

- **Docker:** Focuses on building, packaging, and running isolated containers on a single host.
- **Kubernetes:** An enterprise orchestrator for managing multi-host container clusters, self-healing, automated scaling, and traffic routing.
- **Synergy:** Docker packages application software into standard image formats, while Kubernetes schedules and maintains those container images in production.

02 / Deploy Workload — Steps 1 to 3 (Manifests & Apply)

Steps 1–3: Authored `deployment.yaml` (`nginx:1.27`, 2 replicas) and `service.yaml` (NodePort `30080`), applied manifests using `kubectl apply`, and verified Pods and Service created successfully.

```
ubuntu@master-node:~/day15-kubernetes$ cat deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
ubuntu@master-node:~/day15-kubernetes$ cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
ubuntu@master-node:~/day15-kubernetes$ kubectl apply -f deployment.yaml
deployment.apps/nginx-deployment created
ubuntu@master-node:~/day15-kubernetes$ kubectl apply -f service.yaml
service/nginx-service created
ubuntu@master-node:~/day15-kubernetes$ kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   2/2     2            2           53s
ubuntu@master-node:~/day15-kubernetes$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE         NOMINA
TED NODE   READINESS GATES
nginx-deployment-7c977bd7bb-lf4cg   1/1     Running   0          90s   10.244.1.2   slave-node   <none>
           <none>
nginx-deployment-7c977bd7bb-tcvs5   1/1     Running   0          90s   10.244.1.3   slave-node   <none>
           <none>
ubuntu@master-node:~/day15-kubernetes$ kubectl get services
NAME            TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes      ClusterIP   10.96.0.1      <none>        443/TCP        19h
nginx-service   NodePort    10.109.39.53   <none>        80:30080/TCP   90s
```

03 / Deep Inspection — Step 4 Part 1 (Describe Resources)

Step 4 (Part 1): Inspected resource state using `kubectl describe deployment nginx-deployment` and `kubectl describe service nginx-service`, validating endpoints (`10.244.1.2:80`, `10.244.1.3:80`).

```
ubuntu@master-node:~/day15-kubernetes$ kubectl describe deployment nginx-deployment
Name:                   nginx-deployment
Namespace:              default
CreationTimestamp:      Sat, 17 Oct 2026 14:34:06 +0000
Labels:                 app=nginx
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=nginx
Replicas:               2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx
  Containers:
   nginx:
    Image:        nginx:1.27
    Port:         80/TCP
    Host Port:    0/TCP
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   nginx-deployment-7c977bd7bb (2/2 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  20m   deployment-controller  Scaled up replica set nginx-deployment-7c977b
ubuntu@master-node:~/day15-kubernetes$ kubectl describe service nginx-service
Name:                     nginx-service
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 app=nginx
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.109.39.53
IPs:                      10.109.39.53
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  30080/TCP
Endpoints:                10.244.1.2:80,10.244.1.3:80
Session Affinity:         None
External Traffic Policy:  Cluster
```

03 / Deep Inspection — Step 4 Part 2 (Cluster Events)

Step 4 (Part 2): Tracked lifecycle and scheduling events using `kubectl get events --sort-by=.metadata.creationTimestamp` to analyze scheduling, node registration, and Pod creation.

```
ubuntu@master-node:~/day15-kubernetes$ kubectl get events --sort-by=.metadata.creationTimestamp
LAST SEEN   TYPE      REASON                   OBJECT                                  MESSAGE
44m         Normal    Starting                 node/slave-node                         Starting kubel
44m         Warning   InvalidDiskCapacity      node/slave-node                         invalid capaci
 filesystem
44m         Normal    NodeAllocatableEnforced  node/slave-node                         Updated Node A
mit across pods
44m         Normal    NodeHasSufficientMemory  node/slave-node                         Node slave-nod
ow: NodeHasSufficientMemory
44m         Normal    NodeNotReady             node/slave-node                         Node slave-nod
ow: NodeNotReady
44m         Warning   Rebooted                 node/slave-node                         Node slave-nod
booted, boot id: f2d01a8d-c6d4-41c8-aa08-cfaae364a199
44m         Normal    NodeHasSufficientPID     node/slave-node                         Node slave-nod
ow: NodeHasSufficientPID
44m         Normal    NodeHasNoDiskPressure    node/slave-node                         Node slave-nod
ow: NodeHasNoDiskPressure
44m         Normal    Starting                 node/slave-node
44m         Normal    Starting                 node/master-node
44m         Normal    NodeHasSufficientMemory  node/master-node                        Node master-no
now: NodeHasSufficientMemory
44m         Normal    Starting                 node/master-node                        Starting kubel
44m         Normal    NodeHasNoDiskPressure    node/master-node                        Node master-no
now: NodeHasNoDiskPressure
44m         Normal    NodeAllocatableEnforced  node/master-node                        Updated Node A
mit across pods
44m         Warning   InvalidDiskCapacity      node/master-node                        invalid capaci
 filesystem
44m         Normal    NodeHasSufficientPID     node/master-node                        Node master-no
now: NodeHasSufficientPID
44m         Normal    NodeReady                node/slave-node                         Node slave-nod
ow: NodeReady
43m         Normal    RegisteredNode           node/slave-node                         Node slave-nod
stered Node slave-node in Controller
43m         Normal    RegisteredNode           node/master-node                        Node master-no
istered Node master-node in Controller
34m         Normal    Starting                 node/slave-node                         Starting kubel
34m         Warning   InvalidDiskCapacity      node/slave-node                         invalid capaci
 filesystem
34m         Normal    NodeHasSufficientMemory  node/slave-node                         Node slave-nod
ow: NodeHasSufficientMemory
34m         Normal    NodeHasNoDiskPressure    node/slave-node                         Node slave-nod
ow: NodeHasNoDiskPressure
34m         Normal    NodeHasSufficientPID     node/slave-node                         Node slave-nod
ow: NodeHasSufficientPID
34m         Normal    NodeAllocatableEnforced  node/slave-node                         Updated Node A
mit across pods
32m         Normal    Scheduled                pod/nginx-deployment-7c977bd7bb-lf4cg   Successfully a
lt/nginx-deployment-7c977bd7bb-lf4cg to slave-node
32m         Normal    Scheduled                pod/nginx-deployment-7c977bd7bb-tcvs5   Successfully a
lt/nginx-deployment-7c977bd7bb-tcvs5 to slave-node
32m         Normal    SuccessfulCreate         replicaset/nginx-deployment-7c977bd7bb  Created pod: n
nt-7c977bd7bb-tcvs5
32m         Normal    SuccessfulCreate         replicaset/nginx-deployment-7c977bd7bb  Created pod: n
nt-7c977bd7bb-lf4cg
32m         Normal    ScalingReplicaSet        deployment/nginx-deployment             Scaled up repl
-deployment-7c977bd7bb to 2
```

03 / Deep Inspection — Step 4 Part 3 (Container Logs)

Step 4 (Part 3): Fetched active container logs via `kubectl logs deployment/nginx-deployment` to verify initialization scripts and worker process readiness.

```
-deployment-7c977bd7bb to 2
32m         Normal    Pulling                 pod/nginx-deployment-7c977bd7bb-tcvs5    Pulling image
32m         Normal    Pulling                 pod/nginx-deployment-7c977bd7bb-lf4cg    Pulling image
32m         Normal    Started                 pod/nginx-deployment-7c977bd7bb-lf4cg    Started contai
32m         Normal    Pulled                  pod/nginx-deployment-7c977bd7bb-tcvs5    Successfully p
nginx:1.27" in 224ms (5.346s including waiting)
32m         Normal    Started                 pod/nginx-deployment-7c977bd7bb-tcvs5    Started contai
32m         Normal    Created                 pod/nginx-deployment-7c977bd7bb-lf4cg    Created contai
32m         Normal    Pulled                  pod/nginx-deployment-7c977bd7bb-lf4cg    Successfully p
nginx:1.27" in 5.155s (5.155s including waiting)
32m         Normal    Created                 pod/nginx-deployment-7c977bd7bb-tcvs5    Created contai
20m         Normal    Starting                node/master-node                         Starting kubel
20m         Normal    NodeAllocatableEnforced node/master-node                         Updated Node A
mit across pods
20m         Normal    NodeNotReady            node/master-node                         Node master-no
now: NodeNotReady
20m         Normal    NodeHasSufficientPID    node/master-node                         Node master-no
now: NodeHasSufficientPID
20m         Normal    NodeHasNoDiskPressure   node/master-node                         Node master-no
now: NodeHasNoDiskPressure
20m         Normal    NodeHasSufficientMemory node/master-node                         Node master-no
now: NodeHasSufficientMemory
20m         Warning   InvalidDiskCapacity     node/master-node                         invalid capaci
 filesystem
20m         Normal    NodeReady               node/master-node                         Node master-no
now: NodeReady
ubuntu@master-node:~/day15-kubernetes$ kubectl logs deployment/nginx-deployment
Found 2 pods, using pod/nginx-deployment-7c977bd7bb-lf4cg
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/17 14:34:12 [notice] 1#1: using the "epoll" event method
2026/10/17 14:34:12 [notice] 1#1: nginx/1.27.5
2026/10/17 14:34:12 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14)
2026/10/17 14:34:12 [notice] 1#1: OS: Linux 7.0.0-1006-aws
2026/10/17 14:34:12 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/10/17 14:34:12 [notice] 1#1: start worker processes
2026/10/17 14:34:12 [notice] 1#1: start worker process 29
2026/10/17 14:34:12 [notice] 1#1: start worker process 30
```

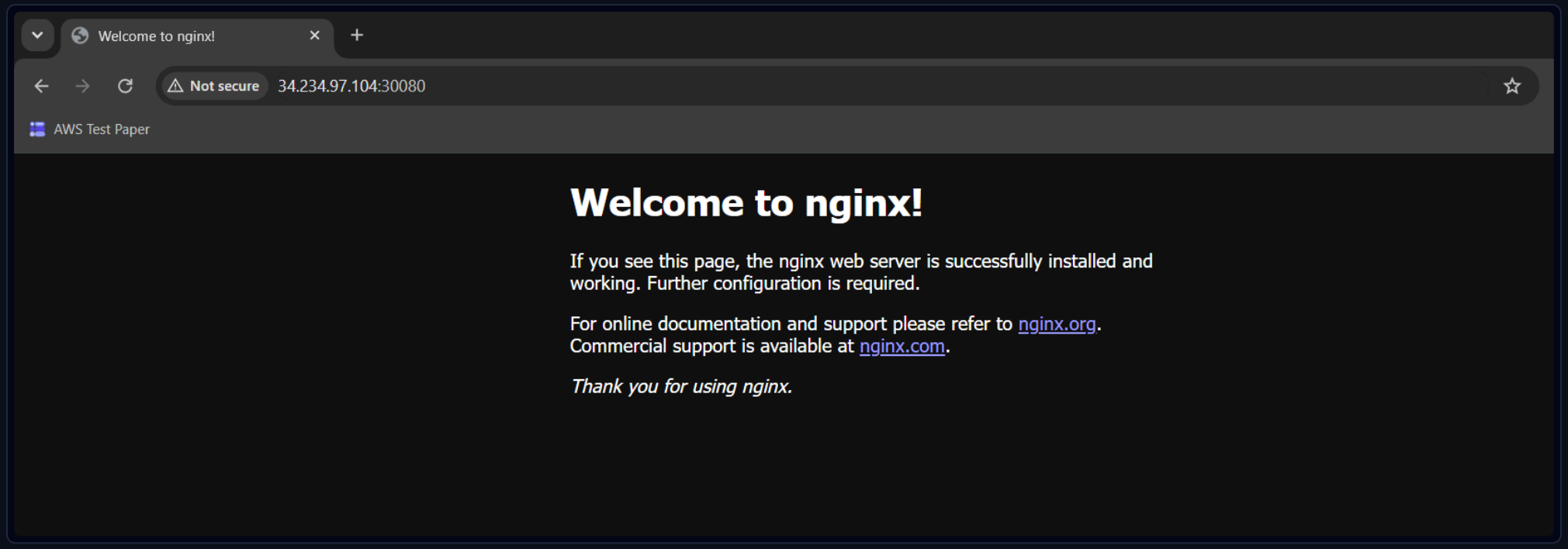
04 / Scale Workload — Step 5 (Scale Deployment to 3 Replicas)

Step 5: Scaled deployment using `kubectl scale deployment nginx-deployment --replicas=3` and verified 3/3 Pods achieved `Running` status.

```
ubuntu@master-node:~/day15-kubernetes$ kubectl scale deployment nginx-deployment --replicas=3
deployment.apps/nginx-deployment scaled
ubuntu@master-node:~/day15-kubernetes$ kubectl get deployment nginx-deployment
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   3/3     3            3           39m
ubuntu@master-node:~/day15-kubernetes$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-7c977bd7bb-lf4cg   1/1     Running   0          39m
nginx-deployment-7c977bd7bb-rsqbg   1/1     Running   0          27s
nginx-deployment-7c977bd7bb-tcvs5   1/1     Running   0          39m
ubuntu@master-node:~/day15-kubernetes$ kubectl get nodes -o wide
NAME          STATUS   ROLES           AGE   VERSION    INTERNAL-IP     EXTERNAL-IP   OS-IMAGE
master-node   Ready    control-plane   19h   v1.29.15   172.31.23.29    <none>        Ubuntu 26.04 LTS
slave-node    Ready    <none>          19h   v1.29.15   172.31.27.110   <none>        Ubuntu 26.04 LTS
```

04 / Scale & Verify — Step 6 Part 1 (Browser Access via NodePort)

Step 6 (Part 1): Accessing the Nginx web server directly in browser via Public Node IP on NodePort `30080` (`http://34.234.97.104:30080`).



04 / Scale & Verify — Step 6 Part 2 (Port-Forwarding & Curl Test)

Step 6 (Part 2): Executed `kubectl port-forward service/nginx-service 8080:80` and verified HTTP response using `curl http://localhost:8080` in a secondary terminal session.

```
ubuntu@master-node:~/day15-kubernetes$ kubectl port-forward service/nginx-service 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

```
ubuntu@master-node:~$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
```

05 / Troubleshooting & Self-Healing — Step 7 (Pod Deletion Test)

Step 7: Demonstrated self-healing by deleting individual Pods using `kubectl delete pod `. The Deployment controller instantly detected the discrepancy against the desired replica count and orchestrated recreation.

```
ubuntu@master-node:~/day15-kubernetes$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-7c977bd7bb-lf4cg   1/1     Running   0          4h29m
nginx-deployment-7c977bd7bb-rsqbg   1/1     Running   0          3h49m
nginx-deployment-7c977bd7bb-tcvs5   1/1     Running   0          4h29m
ubuntu@master-node:~/day15-kubernetes$ kubectl delete pod^C
ubuntu@master-node:~/day15-kubernetes$ kubectl delete pod nginx-deployment-7c977bd7bb-lf4cg
pod "nginx-deployment-7c977bd7bb-lf4cg" deleted
ubuntu@master-node:~/day15-kubernetes$ kubectl delete pod nginx-deployment-7c977bd7bb-rs
Error from server (NotFound): pods "nginx-deployment-7c977bd7bb-rs" not found
ubuntu@master-node:~/day15-kubernetes$ kubectl delete pod nginx-deployment-7c977bd7bb-rsqbg
pod "nginx-deployment-7c977bd7bb-rsqbg" deleted
ubuntu@master-node:~/day15-kubernetes$ kubectl delete pod nginx-deployment-7c977bd7bb-tcvs5
pod "nginx-deployment-7c977bd7bb-tcvs5" deleted
```

05 / Troubleshooting & Self-Healing Architecture

Problem / Scenario	Diagnostic Command	Resolution & Key Learning
Pod is Pending	`kubectl describe pod <pod-name>`	Inspect events for resource constraints or node scheduling failures.
ImagePullBackOff	`kubectl describe pod` / `kubectl logs`	Verify image tag (`nginx:1.27`) and network connectivity to registry.
Service has no endpoints	`kubectl get pods --show-labels`	Ensure Service selector labels (`app: nginx`) strictly match Pod labels.
Self-Healing Test	`kubectl delete pod <pod-name>`	Kubernetes Deployment controller automatically provisions a new replacement pod to maintain the desired replica count.

06 / Deliverable & Review Criteria Checklist

Repository Structure (`day15-kubernetes/`)

- **deployment.yaml:** Configured with 2 (scaled to 3) replicas and `nginx:1.27`.
- **service.yaml:** Configured as NodePort routing to port 80 on port 30080.
- **README.md:** Comprehensive documentation of cluster setup, concepts, and commands.
- **evidence/:** Complete screenshots for nodes, pods, deployment, scaling, and access tests.

Security & Cleanup Hygiene

- **Credentials Safety:** No kubeconfig files, tokens, or cloud credentials uploaded.
- **Cleanup Commands:**

```
kubectl delete -f service.yaml
kubectl delete -f deployment.yaml
```

07 / Interview Readiness & Architecture Q&A

1. What is a Pod and why not run containers directly?

- A Pod is the smallest execution unit in Kubernetes, wrapping one or more co-located containers that share network namespaces, IP addresses, and storage volumes.

2. How does a Service route traffic to Pods?

- Services use `label selectors` (e.g., `app: nginx`) to discover running Pods and dynamically update endpoint objects for layer 4 load balancing.

3. What role does a Deployment play?

- Deployments manage ReplicaSets, enabling declarative updates, seamless zero-downtime rolling updates, automated rollbacks, and self-healing.

4. Difference between NodePort and ClusterIP?

- `ClusterIP` exposes the service internally within the cluster, whereas `NodePort` opens a static port on every worker node's external IP to route external traffic.